

Predicting satisfaction of airline passengers using classification

Lamija Hibović

International University of Sarajevo
Faculty of Engineering and
Natural Sciences
Sarajevo, Bosnia and Herzegovina
lhibovic@student.ius.edu.ba

Selma Smajić

International University of Sarajevo
Faculty of Engineering and
Natural Sciences
Sarajevo, Bosnia and Herzegovina
selmasmajic@student.ius.edu.ba

Associate Professor

Dr. Emine Yaman
International University of Sarajevo
Faculty of Engineering and
Natural Sciences
Sarajevo, Bosnia and Herzegovina
eyaman@ius.edu.ba

This research paper is exploring the application of classification data mining algorithms on specific data. The data set used in this paper is open and available to everyone, it contains several attributes that are measuring the overall quality of the flight. The main aim of the paper is to explore the effect of different flight attributes on the overall satisfaction of the passenger.

Keywords — flight, satisfaction, classification, algorithm

I. INTRODUCTION

Data mining is a branch of artificial intelligence and it is widely applicable to a variety of everyday problems. Concepts and algorithms used in data mining are comprehensive and elaborative. It is common to find different applications of data mining in areas such as the airline industry. The application goes from the aviation were based on different variables we can calculate the amount of fuel needed for flights[1] to passenger satisfaction where we can calculate passenger satisfaction based on particular factors. This paper focuses on calculating passenger satisfaction of a particular airline using classification. By definition, classification is a process of classifying or systematic arrangement in predefined groups or categories.[2] Classification belongs to a predictive method where we are using some variables to predict features values of other variables.[3] Particularly, we are using multiple different variables related to the quality of flight and whole flight organization to measure final passenger satisfaction.

Several research papers are examining this problem, for example, paper *A data mining approach to analyze customer satisfaction from airline tweets*[4] written by Sachin Kumar & Mikhail Zimblar they used data from tweets to get passengers feedback they used SVM (support vector machine) and several ANN (artificial neural network) to create their classification model, while we used KNN, Naïve Bayes, decision tree-based methods and rule-based methods. However, there is a paper *Sentiment classification system of Twitter data for US airline service analysis* [5] that is using a decision tree, random forest, Gaussian Naïve Bayes, SVM, KNN, logistic regression, and AdaBoost. They trained the classifiers on 80% of the data and the remaining data were used for testing. They classified the tweets into three categories of sentiments. They mentioned that logistic regression, AdaBoost, random forest, and SVM performed well on the model with an accuracy of more than 80%.

II. MATERIALS AND METHODS

A. About dataset

Dataset used in this paper is called *Airline Passenger Satisfaction*[6]. It consists of 25 variables with 26025 observations. A list of all variables can be seen in Image 1, there are 20 numeric variables and five-character variables. Variable satisfaction is an output variable which is determining whether the passenger is “neutral or dissatisfied” or “satisfied”. From a total of 25025 observations, 14600 were “neutral or dissatisfied” and 11425 were “satisfied”, this makes our data set balanced.

```
-- Data Summary -----
Name      values
Number of rows      26025
Number of columns    25

Column type frequency:
character      5
numeric       20

Group variables      none

-- Variable type: character -----
# A tibble: 5 x 8
  skin_variable n_missing complete_rate min max empty n_unique whitespace
<chr>          <dbl>      <dbl>   <dbl> <dbl> <dbl> <dbl>      <dbl>
1 gender        0          1      0     6    0      2          0
2 customer_type 0          1      0    17   250  3          0
3 type_of_travel 0          1    15    15    0      2          0
4 class         0          1      3     8    0      3          0
5 satisfaction   0          1      9    23    0      2          0

-- Variable type: numeric -----
# A tibble: 20 x 11
  skin_variable n_missing complete_rate mean sd    q0    q05    q50    q75    q90    hist
<dbl>          <dbl>      <dbl>   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 X              0          1 12985.  7513.  0 6479 12985 18491 25979
2 X              0          1 68009.  37600. 17 36176 68322 93752 123887
3 Age            311      0.988    39.6  15.1  7  27  40  51  85
4 Flight.Distance 209      0.996 1195. 1000. 31 424 850. 1747 4989
5 Inflight.wifi.service 0          1  2.73  1.34  0  2  3  4  5
6 Departure.Arrival.time.convenient 0          1  3.05  1.53  0  2  3  4  5
7 Ease.of.online.booking 0          1  2.76  1.45  0  2  3  4  5
8 Date.location 15647      0.399  2.95  1.36  1  2  3  4  5
9 Food.and.drink 0          1  3.22  1.33  0  2  3  4  5
10 Online.boarding 0          1  3.26  1.36  0  2  3  4  5
11 Seat.comfort 0          1  3.45  1.32  1  2  3  4  5
12 Inflight.entertainment 0          1  3.36  1.34  0  2  3  4  5
13 On.board.service 0          1  3.39  1.28  0  2  3  4  5
14 Legroom.service 0          1  3.35  1.32  0  2  3  4  5
15 Baggage.handling 0          1  3.63  1.18  1  3  4  5  6
16 Checkin.service 0          1  3.31  1.27  1  3  4  5  6
17 Inflight.service 0          1  3.65  1.18  0  3  4  5  6
18 Cleanliness 0          1  3.29  1.32  0  2  3  4  5
19 Departure.Delay.in.Minutes 0          1 14.3  37.4  0  0  0 12 1138
20 Arrival.Delay.in.Minutes 83      0.997 14.7  37.5  0  0  0 12 1138
```

Image 1: Data summary created in R using skim() function from skimr librar[7]

Variables such as Age, Gender, Customer. Type and Class are describing passengers, all other variables are describing flight and flight-related factors. It is interesting to mention that there are 13 variables where passengers rated flight factors such as Seat. comfort, Cleanliness, and similar are rated from 0 to 5 so these variables were easier to manipulate. In Image 1, we can see that majority of these variables have a mean from 3.20 to 3.60, we can see on histograms that the majority of grades were 4 or 5.

Before data pre-processing, we have created boxplots for continuous variables: age, flight distance, departure delay in minutes, and arrival date in minutes

(Figures 1-4). Looking at the box plot in Figure 1 we can see that a bigger proportion of data falls into the neutral or dissatisfied category, rather than in the satisfied. The median of satisfied customers is 43 years while the median of neutral or dissatisfied customers is 37 years.

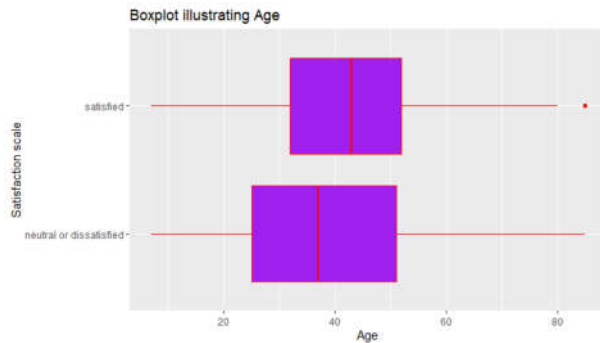


Figure 1: Age vs satisfaction boxplot

When it comes to flight distance more data tend to fall in satisfied passengers in flights from 500 km to 2500 km. While dissatisfied customers are on shorter flights or very long ones. This could also affect their grading since in short flights passengers cannot fully experience airline offers and long flights can be stressful themselves so grades can be low regardless.

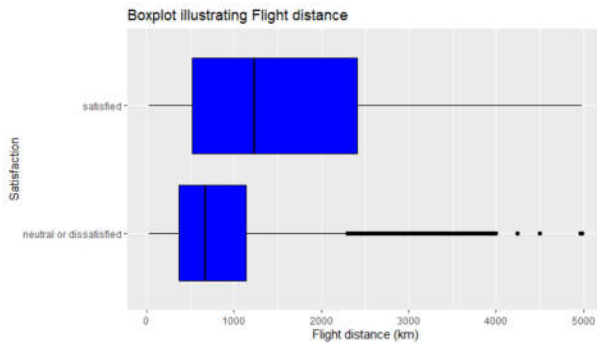


Figure 2: Flight distance vs satisfaction boxplot

We can see that Figure 3 has a lot of outliers, meaning that departure delays are not that often and that around 75% of data rotates around zero, however, exceptions can happen and big delays are presented as outliers. Comparing satisfied and dissatisfied we can see that boxplot is thinner for satisfied passengers, which is logical that as smaller delay is passenger will be more satisfied.

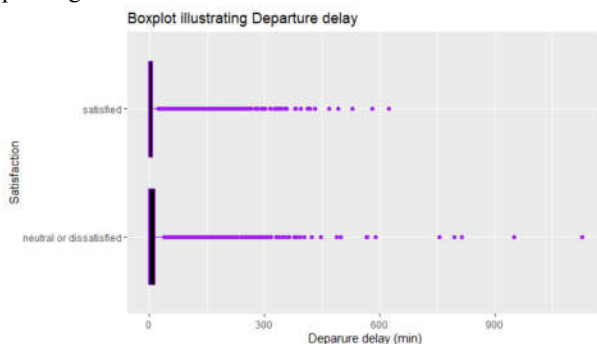


Figure 3: Departure delay vs satisfaction boxplot

We can see that Figure 4 has a lot of outliers, meaning that arrival delays are not that often and that around 75% of data rotates around zero, however, exceptions can happen and big delays are presented as outliers. Comparing satisfied and dissatisfied we can see that boxplot is thinner for satisfied passengers, which is logical that as smaller delay is passenger will be more satisfied.

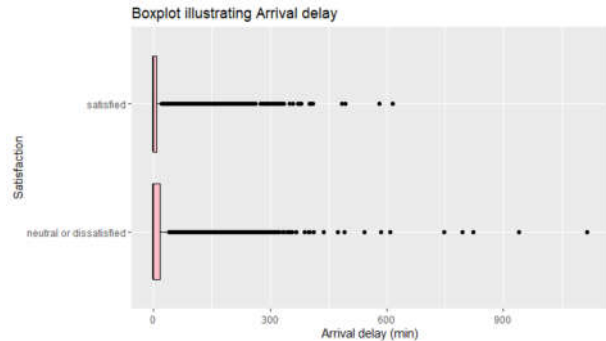


Figure 4: Arrival delay vs satisfaction boxplot

To further analyze the data, we compared some categorical values and levels of satisfaction using bar plots. (Figures 5-7) In Figure 5, we can see that loyal passengers tend to be more satisfied than disloyal ones. This could lead us to question the validity of the data since we do not know the criteria for assessing the loyalty of the customer. Overall satisfaction in disloyal passengers is 0.248, while satisfaction in loyal customers is 0.477.

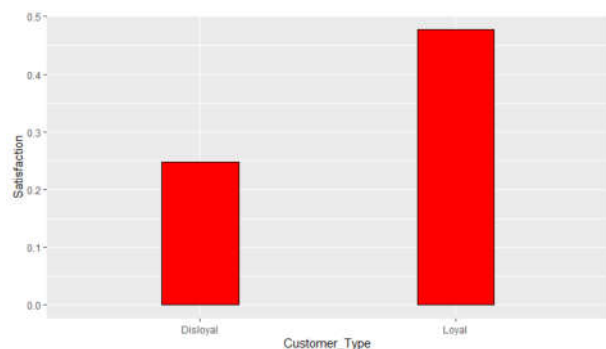


Figure 5: Customer type vs satisfaction bar plot

In Figure 6 Value of male satisfaction is 0.443 and Female satisfaction is 0.435 so we can say that satisfaction does not depend on the gender of a passenger.

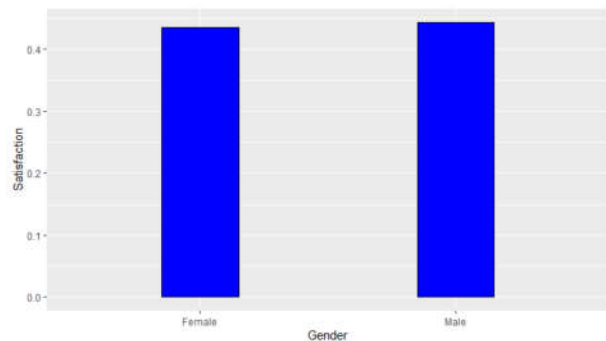


Figure 6: Gender vs satisfaction bar plot

Observing Figure 7 it is obvious that class can highly impact the satisfaction of a passenger, which is logical in a way that tickets for business class are more expensive therefore business class passengers are of higher priority to the airline. Satisfaction of business class is 0.695 while for Eco is 0.194 and for Eco Plus is 0.248. Even though it's logical that passengers in business class have higher satisfaction, the difference between Economy class and Bussiness should not be so large.

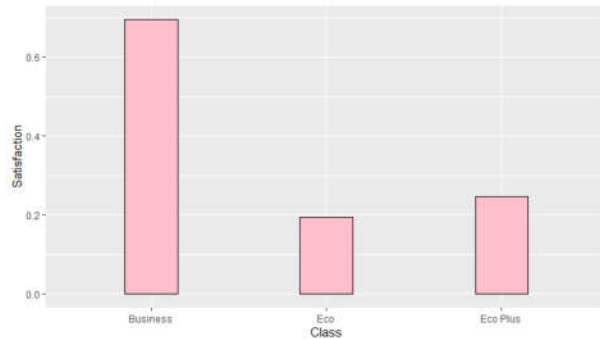


Figure 7: Class vs satisfaction bar plot

Figure 8 represents box plots of all variables in the data set that stored passengers' grades from 0 to 5. Observing the box plot where the black dot is presenting the mean of the data we can see that is going around number 3 which makes it average satisfaction level of all of these features.

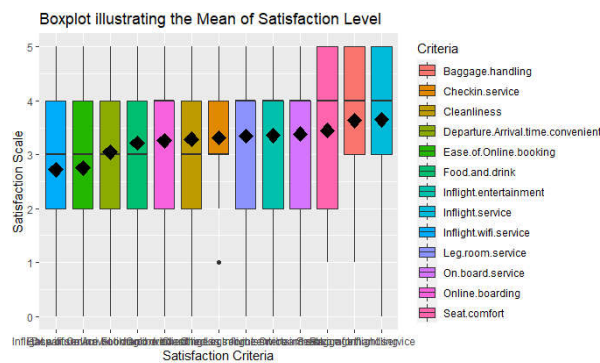


Figure 8: Mean satisfaction level of gathered variables

B. PREPROCESSING

The first step in pre-processing was to remove variables that do not carry valuable or significant information for our model creation, therefore we have removed variables X and id. These variables were different for every observation and could only complicate models. Next, we obtained duplicate data and removed it from the data set since it could negatively affect models. To handle categorical data, we converted all data to numerical values since we used the kNN algorithm in our research and kNN is a distance-based algorithm, therefore, it requires numerical data. Data was also normalized since for example flight distance has a huge range it would affect our results significantly, therefore, normalization was a must. It is important to mention that in all models we used scaled and

numerical data to be able to compare and evaluate our results. Data is not imbalanced therefore we could make a split in the following way: 80% for training and 20% for testing.

In the phase of handling categorical data, we checked for each attribute its possible values. For Customer Type, we found out that missing values are not encoded as NA but as space (""). We had to recode it and this made it easier to handle missing values. If we did not examine all attributes, we would have not discovered this possibly human error.

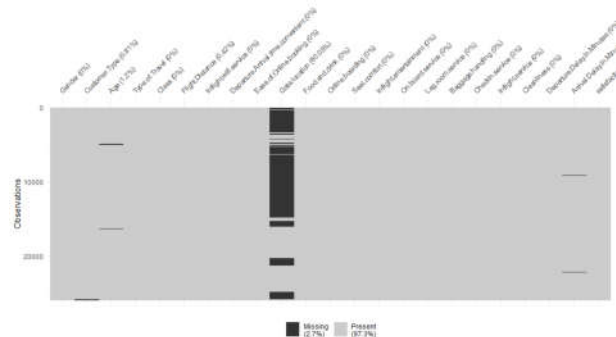


Image 2: Missing values in the data set

As we can see in Image 2 missing values in our data set are presented in the following variables Customer. Type, Age, Flight. Distance, Gate. location and Arrival. delay, however from all of the Gate. location is highly affected since 60.05% of data is missing therefore we decided to exclude this variable from our dataset since it would create our models biased and not so representative. For other missing values that are categorical, we used mode imputation due to the low percentage of missing data. To prove that a mode imputation did a good job, we compared distribution after imputation of missing values and distribution of the original data which can be seen in Figure 9. During mode imputation, we came across difficulty since we have tried to use the predefined mode() function on variable Customer. Type, but it treated NA as numeric.

Further on, when it comes to numerical data, we used median imputation since the distribution of this variable is skewed and we could not use mean imputation that is sensitive to outliers. For the Age attribute, we use regular median imputation. However, we checked 5 numbers summary for genders that turned out almost identical. Figure 11 shows the effectiveness of this method, there is a small peak at a value of 40 in the distribution of data without missing values since the median is 40. In this case, this difference is acceptable since there is a small number of missing values. However, if the percentage of missing values were higher, this method would not be successful, we would have disturbed the distribution. For Flight Distance, we used similar case median imputation since different types of customers have a different distances of flights. Based on the 5 number summary, we realized that customers of the

business class have longer flights while customers of the eco class have smaller distance flights.

Figure 10 proves that similar case median imputation did a good job since the distribution of original data and data after imputation of missing values are almost identical. To deal with missing values, we also used a correlation matrix. We have found that Arrival Delay in Minutes and Departure Delay in Minutes are highly correlated, to be precise 0.9645018. In other words, keeping both attributes would be redundant and it may negatively affect our results since Arrival Delay in Minutes has missing data and it is low correlated to the target, this attribute is dropped.

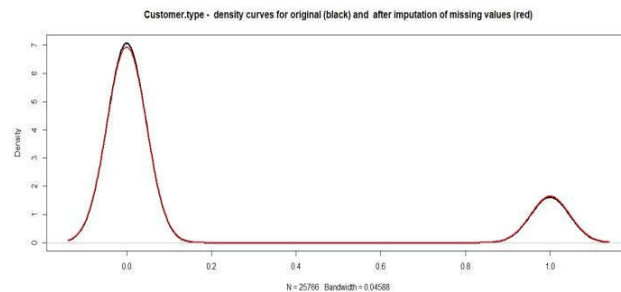


Figure 9: Customer type

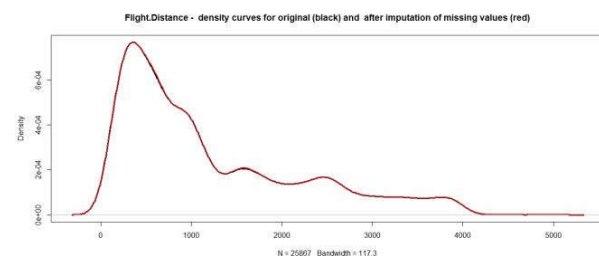


Figure 10: Flight distance

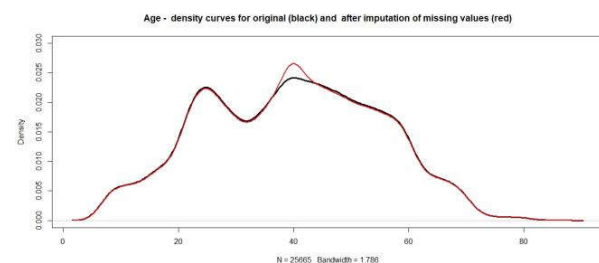


Figure 11: Age

C. FEATURE SELECTION

When it comes to feature selection we used Bourta() function. Boruta is a feature selection algorithm, it adds randomness to the given data set by creating shuffled copies of all features. Then, it trains a random forest classifier on the extended data set and applies a feature importance measure (the default is Mean Decrease Accuracy) to evaluate the importance of each feature where higher means more important. At every iteration, it checks whether a real feature has higher importance than the best of its shadow features and constantly removes features that are deemed highly unimportant. Finally, the algorithm stops either when

all features get confirmed or rejected or it reaches a specified limit of random forest runs. We got that all features are important, as can be seen on *Image 3*, *Image 4*, and *Image 5*. Therefore we decided to add a random variable to check the validity of these results and Bourta() successfully recognized this variable. This can be seen in *Image 6*, *Image 7*, and *Image 8*.

```
> boruta_output <- Boruta(satisfaction ~ ., data = airline, dotrace = 2)
1. run of importance source...
2. run of importance source...
3. run of importance source...
4. run of importance source...
5. run of importance source...
6. run of importance source...
7. run of importance source...
8. run of importance source...
9. run of importance source...
10. run of importance source...
11. run of importance source...
After 11 iterations, +6.3 mins:
confirmed 20 attributes: Age, Baggage.handling, Checkin.service, Class, Cleanliness and 15 more;
no more attributes left.
```

Image 3: Boruta output (no rejected attributes)

```
> print(boruta_output)
Boruta performed 11 iterations in 6.339049 mins.
20 attributes confirmed important: Age, Baggage.handling, Checkin.service, Class, Cleanliness and 15 more;
No attributes deemed unimportant.
```

Image 4: Boruta output (no attributes unimportant)

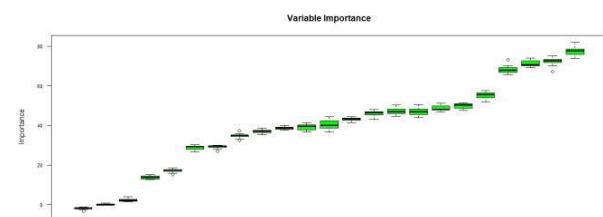


Image 5: All attributes have importance (all green)

```
> boruta_output_1 <- Boruta(satisfaction ~ ., data = airline_irrelevant, dotrace = 2)
1. run of importance source...
2. run of importance source...
3. run of importance source...
4. run of importance source...
5. run of importance source...
6. run of importance source...
7. run of importance source...
8. run of importance source...
9. run of importance source...
10. run of importance source...
11. run of importance source...
12. run of importance source...
Computing permutation importance.. Progress: 95%. Estimated remaining time: 1 seconds.
After 12 iterations, +6.7 mins:
confirmed 20 attributes: Age, Baggage.handling, Checkin.service, Class, Cleanliness and 15 more;
rejected 2 attributes: random_1, random_2;
no more attributes left.
```

Image 6: Boruta output (2 attributes recognized as unimportant)

```
> print(boruta_output_1)
Boruta performed 12 iterations in 6.726841 mins.
20 attributes confirmed important: Age, Baggage.handling, Checkin.service, Class, Cleanliness and 15 more;
2 attributes confirmed unimportant: random_1, random_2;
```

Image 7: All attributes have importance, except two randomly added

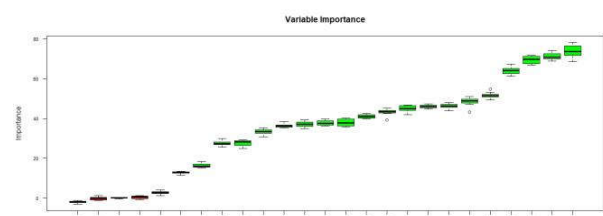


Image 8: Boruta output(2 rejected attributes)

D. ALGORITHMS

1) K-Nearest Neighbors

K-nearest neighbors (kNN) is the first algorithm used in our process, this is a very simple algorithm that is easy to implement, yet it gives solid results.[8] This is a supervised algorithm meaning that it looks into known labeled data to create a function that will predict the new unlabeled output of data. The main idea of kNN is that similar data is near to each

other, therefore this algorithm is a distance-based algorithm. The algorithm will calculate the distance between points on the graph. There are several approaches to calculate distance, some of them are Euclidean distance, Minkowski, and Manhattan distance. Mostly used is Euclidean distance.

Overall advantages of kNN are that is easy to implement, and is versatile so it can be used in classification and regression. However, the disadvantage of this algorithm is that it gets slower as we increase the number of examples or predictors.

This is a supervised algorithm meaning that it looks into known labeled data to create a function that will predict the new unlabelled output of data. The main idea of kNN is that similar data is near to each other, therefore this algorithm is distance-based. The algorithm will calculate the distance between points on the graph. There are several approaches to calculate distance, some of them are Euclidean distance, Minkowski, and Manhattan distance. Mostly used is Euclidean distance. Overall advantages of kNN are that is easy to implement, and is versatile so it can be used in classification and regression. However, the disadvantage of this algorithm is that it gets slower as we increase the number of examples or predictors. As a resampling method, we used cross-validations, which tests model performance in different iterations.

First, we calculated the square root of a number of observations to get K, and we created models for k=5, 50, 100, and the square root of 161. Since performances are going down when k is bigger, we did 10 fold cv for 3 different fits of the range of k. From 1 to 15, then from 45 to 60 and from 100 to 125. These ranges produce the following graphs:

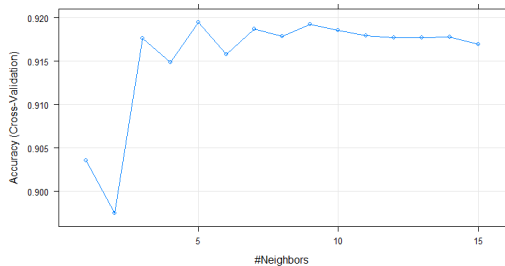


Figure 12: Graph of accuracy for k=1:15

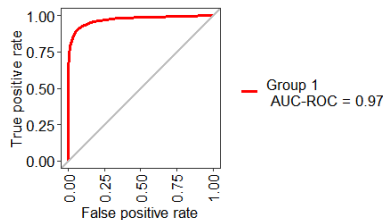


Figure 13: ROC curve for k=1:15

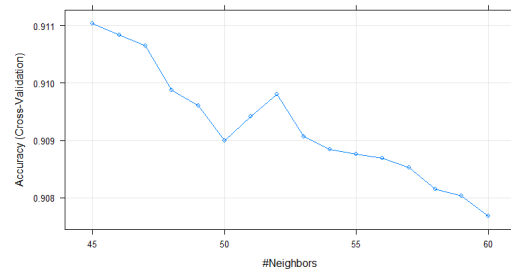


Figure 14: Graph of accuracy for k=45:60

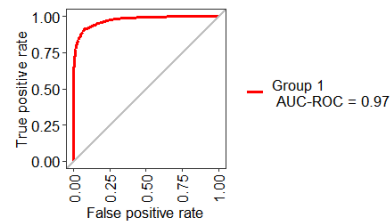


Figure 15: ROC curve for k=45:60

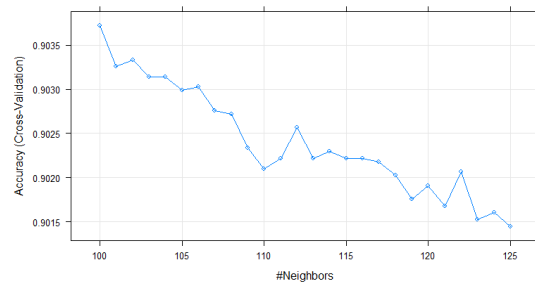


Figure 16: Graph of accuracy for k=100:125

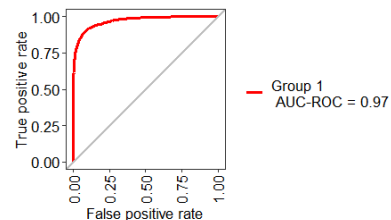


Figure 17: ROC curve for k=100:125

From the graphs, we can conclude that k=5 is the best and we choose k=5 for the model.

2) Random forest

Random Forest is one of the decision trees models and it is commonly used in data mining. It contains several decision trees on different subsets of the data set and it calculates the average to improve the accuracy. It makes it unique since it does not rely on one tree, rather it takes predictions from each tree, and based on the majority of votes it decides the final output. The greater number of trees in the forest leads to higher accuracy and prevents the problem of overfitting. In R we used library(randomForest). As we can see in Image 9 number of trees created in this model was 500. Unfortunately, the plot was too crowded with trees, so we did not include them in the report. The problem that we have encountered while using this library is that randomForest

was constantly making a regression instead of classification. This happened because initially we got results as values between 0 and 1 and we did this by a setting condition that if the output is less than 0,6 we output 0 or if it is grater we output 1. However later on we have found that our class label has to be a factor, not an int value of 0 or 1 to make a classification.

```
> #creating random forest model
> model_rf <- randomForest(as.factor(satisfaction) ~ ., data = train)
> print(model_rf)

Call:
randomForest(formula = as.factor(satisfaction) ~ ., data = train)
Type of random forest: classification
Number of trees: 500
No. of variables tried at each split: 4

OOB estimate of error rate: 4.4%
Confusion matrix:
  0   1 class.error
0 11304 354 0.03036541
1   560 8562 0.06139005
```

Image 9: Random Forest output

3) Decision tree-based algorithms

The decision tree baes algorithms are one of the most used techniques in data mining. The main advantage of these techniques is that they can be used with almost all types of data. The decision tree techniques use the divide and conquer method, which is a recursive approach, and it means that data is separated into smaller subsets. Models in decision tree techniques are presented as a tree.

a) C5.0 decision trees

The decision tree works on the divide and conquers technique because it uses values of variables to split data into smaller values. For the decision tree, we used the C5.0 package and function for the tree and it relies on entropy which is a measure of purity. If the segments of the data contain only a single class, they are considered pure. The tree is purging in a way that the algorithm finds labels with the lowest purity and that is our initial note, next split will be on a node that has the next lowest purity, and so on. Since it can be with and without rules, we created two models with C5.0, one model is with rules=FALSE, and another one is with rules=TRUE.

Default class: 0

Evaluation on training data (20780 cases):

```
-----
Rules
-----
No      Errors
-----
58      861( 4.1%)  <<

(a)      (b)      <-classified as
-----
11303    355      (a): class 0
506      8616     (b): class 1
```

Attribute usage:

```
99.90% Inflight.wifi.service
46.62% online.boarding
46.13% Type.of.Travel
45.97% Customer.Type
43.41% Inflight.entertainment
42.72% Seat.comfort
40.77% Inflight.service
40.24% checkin.service
38.41% Cleanliness
37.46% Baggage.handling
32.27% Leg.room.service
24.02% Class
21.36% Departure.Arrival.time.convenient
18.11% On.board.service
16.26% Departure.Delay.in.Minutes
13.44% Ease.of.Online.booking
1.74% Food.and.drink
0.26% Flight.Distance
0.15% Age
```

Time: 0.8 secs

Image 10: Ruled C5.0

Evaluation on training data (20780 cases):

```
-----
Decision Tree
-----
Size      Errors
-----
153      820( 3.9%)  <<

(a)      (b)      <-classified as
-----
11370    288      (a): class 0
532      8590     (b): class 1
```

Attribute usage:

```
100.00% Inflight.wifi.service
96.86% online.boarding
84.41% Type.of.Travel
49.35% Customer.Type
43.66% Cleanliness
40.17% Seat.comfort
39.85% checkin.service
32.52% Baggage.handling
25.68% Departure.Arrival.time.convenient
24.17% Class
22.16% Inflight.service
19.02% Inflight.entertainment
16.28% On.board.service
7.22% Leg.room.service
1.82% Age
1.47% Food.and.drink
0.77% Ease.of.Online.booking
0.51% Departure.Delay.in.Minutes
0.33% Gender
0.08% Flight.Distance
```

Time: 0.4 secs

Image11: Not ruled C5.0

If we take a look at Image 10 and 11 we can see that both types of algorithms recognized variable Inflight.wifi.service as a feature with the lowest purity. The next two features are the same but further on they differ.

b) Recursive partitioning

After researching the ways of plotting the decision tree, we've stumbled upon a *rpart* and *rpart.plot* packages, which gave us the possibility to plot a beautiful decision tree. In general, recursive partitioning is seen more as a statistical method to analyze sets with many variables. In R, the *rpart()* function uses similar logic as C5.0(), just for the measure of impurity it uses GINI instead of entropy. [9]

```
Call:
rpart(formula = satisfaction ~ ., data = train, method = "class")
n= 20780

cp      nsplit rel error   xerror   xstd
1 0.50657750  0 1.0000000 1.0000000 0.007842314
2 0.13374260  1 0.4934225 0.4934225 0.006509618
3 0.04220566  2 0.3596799 0.3596799 0.005762314
4 0.03003727  3 0.3174742 0.3174742 0.005472916
5 0.02444639  4 0.2874370 0.2874370 0.005247320
6 0.01037784  5 0.2629906 0.2629906 0.005049947
7 0.01000000  8 0.2318570 0.2436966 0.004884395

variable importance
online.boarding      Inflight.wifi.service      Seat.comfort Ease.of.online.booking
26                  21                      12                  10
Type.of.Travel      Inflight.entertainment      cleanliness      class
10                   9                        8                   1
checkin.service      Age      Inflight.service
1                    1                      1
```

Image 12: RPART model evaluation

From the summary of the model, it's important to note cp, i.e. the complexity parameter, which is the minimum improvement in the model needed at each node. That is, it adds up the misclassification at every terminal node. As seen in Image 12, the higher the cp, the smaller the error. The cp value is also a stopping parameter, as it helps speed up the search for the best split (the main problem in decision trees), which is why a *rpart* package is a powerful tool for decision trees. 14 Additionally, regarding the variable importance, unlike in the C5.0 models, the *rpart* model shows that Online. boarding has the highest importance, followed by the Inflight.wifi.service, and the tree itself is seen in Figure 18 below.

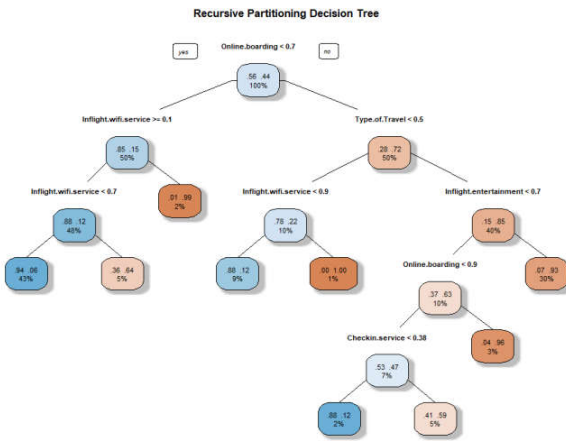


Figure 18: RPART Tree Model

After some more research, we've found out about tree pruning and wanted to see if we'd have different results if we pruned our RPART model. To do that, it's needed to see how does the *cp* value change, and with the help of a function *printcp()*, we could see the error rate for different *cp*'s:

4) Naïve Bayes

Naive Bayes is another data mining algorithm that we used in our project. This algorithm is used for classification based on probabilities. Naive Bayes algorithms use feature variables to calculate the probability of an event. It means that the algorithm calculates the probability of outcomes. In our case, the outcome is customer satisfaction. Customer satisfaction has two possible outcomes: neutral or dissatisfied. Naive Bayes algorithms are fast and efficient and it is easy to implement.

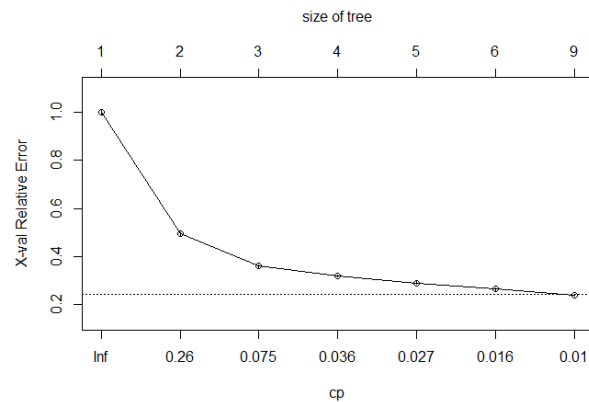


Figure 19: CP plot

Utilizing the function *prune()* from the *rpart* package, we've told R to use best *cp*, and then to plot the pruned RPART tree, which can be seen in Figure 20:

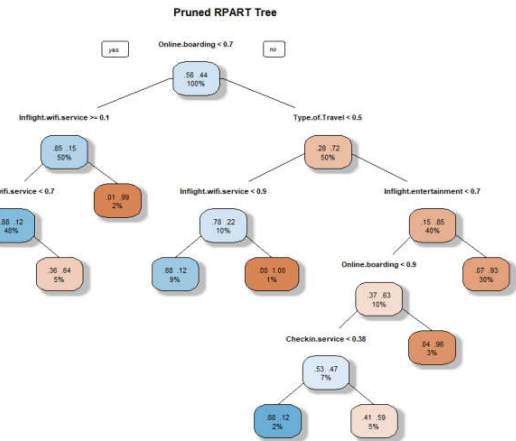


Figure 20: Pruned RPART Tree

```
> # summary of pruned rpart tree
> summary(pruned_rpart)
call:
rpart(formula = satisfaction ~ ., data = train, method = "class")
n = 20780

      cp nsplit rel error   xerror   xstd
1 0.50657750  0 1.0000000 1.0000000 0.007842314
2 0.13374260  1 0.4934225 0.4934225 0.006509618
3 0.04220566  2 0.3596799 0.3596799 0.005762314
4 0.03003727  3 0.3174742 0.3174742 0.005472916
5 0.02444639  4 0.2874370 0.2874370 0.005247320
6 0.01037784  5 0.2629906 0.2629906 0.005049947
7 0.01000000  8 0.2318570 0.2436966 0.004884395

Variable Importance
online.boarding  Inflight.wifi.service  Seat.comfort  Ease.of.online.booking
26                21                    12                10
Type.of.Travel  Inflight.entertainment  Cleanliness      Class
10                9                      8                1
checkin.service  Age                    Inflight.service
1                 1                      1
```

Image 13: Pruned RPART Model Evaluation

Comparing Figures 18 and 20, as well as the summaries (Images 12 and 13), we've concluded that these two models are identical.

5) LOGISTIC REGRESSION

Logistic regression is a powerful supervised data mining algorithm, used primarily for binary classification problems, as the name says (logical) – it's a linear regression, but for classification. The main difference from the "normal" (linear) regression is that the range of the logistic regression is between 0 and 1. Also, logistic regression doesn't require a linear relationship between input and output variables. The best way to think of the logistic regression is that it's like a fitting line. The main reason why we wanted to apply Logistic Regression to our data is to see just how would another algorithm perform.

III. RESULTS AND DISCUSSION

```
> accuracy_table_ordered
naive bayes      random forest      Decision tree C50 ruled      decision tree
0.9958352      0.9581153      0.9543982      0.9501671
knns Decision tree using rpart      Pruned r part      knns0
0.9183449      0.9019665      0.9019665      0.9029239
0.8941949      0.8876380      0.8605155

> specificity_table_ordered
naive bayes      random forest      Decision tree C50 ruled      decision tree
0.9915703      0.9460763      0.9355546      0.9329242
Decision tree using rpart      Pruned r part      knns      knns0
0.9092103      0.9092103      0.8737396      0.8426129
knns100      knns161      logistic regression      0.7851819
0.8298992      0.8185007

> sensitivity_table_ordered
naive bayes      Decision tree C50 ruled      random forest      decision tree
1.0000000      0.9732419      0.9701544      0.9674099
knns      knns0      knns100      knns161
0.9629503      0.9623350      0.9684906      0.9667753
logistic regression Decision tree using rpart      Pruned r part
0.9358491      0.8946827      0.8946827
```

Image 14: Summary of all models

A. RESULTS

To evaluate the data, we have to compare accuracy, sensitivity, and specificity. The sensitivity of a model (also called the true positive rate), measures the proportion of positive examples that were correctly classified. Naïve Bayes has the highest sensitivity (which is 1), then ruled

decision tree and random forest. When it comes to kNN, a model with $k = 5$ has the highest sensitivity, which proves that this value of hyperparameter gives the best performance. RPART scored the lowest, being pruned or not. From the previous data, we can conclude that the Inflight.wifi.service is much more important than Online boarding for the passenger satisfaction, since the models that chose Inflight.wifi.service to be most important have the highest true positive rate.

The specificity of a model (also called the true negative rate), measures the proportion of negative examples that were correctly classified. Naïve Bayes has the highest specificity, then ruled decision tree and random forest, same as in sensitivity. When it comes to kNN, a model with $k = 5$ has the highest specificity which again proves that this value of hyperparameter gives the best performance.

Accuracy is just a fraction of predictions that our model is good. From the table above (Image 14), we can see that Naïve Bayes has the best performance, shortly followed by Random Forest. The random forest has high accuracy, since by its nature as we decrease the number of trees accuracy increases and in our case, we had 500 trees. The weakest performance is by logistic regression.

B. LIMITATIONS

One limitation for our project was insight in variables, for example when it comes to variable Customer. Type, customers were labeled as loyal and disloyal, however, we did not know based on which criteria the airline decided to label them as such. The next limitation is that we had to limit ourselves to several models that will be applied to our data set.

IV. CONCLUSION

Results of the applied algorithms made us understand data mining algorithms more in-depth. This data set was challenging in a way that pre-processing required a lot of knowledge in R and data mining in general. We used mode and median imputation for missing data and we converted all of our data into numerical values to be easily used by models. All data were scaled so that final results can be compared among models. Boruta was used for feature selection and we proved that all features of the data set are as important. Further on in k-NN, we have proven by final results that hyperparameter $k = 5$ shows the best results for this model.

We have concluded that Naïve Bayes had the best performance results, followed by Random Forest and Ruled Decision Tree. Naïve Bayes once again proved that is easy to implement and that is very efficient in multifeatured environments. RPART and Logistic Regression showed weaker performance.

Further work that could be done is the application of oversampling in models where is not applied such as in decision trees which would be bootstrapping.

REFERENCES

- [1] https://www.academia.edu/37849988/Sentimental_Analysis_for_Airline_Twitter_data_Sentimental_Analysis_f_or_Airline_Twitter_data
- [2] <https://dictionary.cambridge.org/dictionary/english/classification>
- [3] P. N. Tan, M. Steinbach, V. Kumar, Introduction to data mining, 2nd ed. Boston, MA, 2005
- [4] <https://journalofbigdata.springeropen.com/articles/10.1186/s40537-019-0224-1>
- [5] Rane A, Kumar A. Sentiment classification system of twitter data for US airline service analysis. In: Proc: 42nd IEEE computer software and applications conference, COMPSAC 2018. Tokyo, Japan. p. 769–73.
- [6] <https://www.kaggle.com/teejmahal20/airline-passenger-satisfaction?select=test.csv>
- [7] <https://cran.r-project.org/web/packages/skimr/vignettes/skimr.html>
- [8] Z. Zang. „Introduction to data mining: k-nearest neighbors“, *Annals of Translational Medicine*, vol. 4, 2016
- [9] <https://cran.r-project.org/web/packages/rpart/vignettes/lonFgintro.pdf>
- [10] Charu C. Aggarwal. *Data Mining: The Textbook*, 2015th Edition